

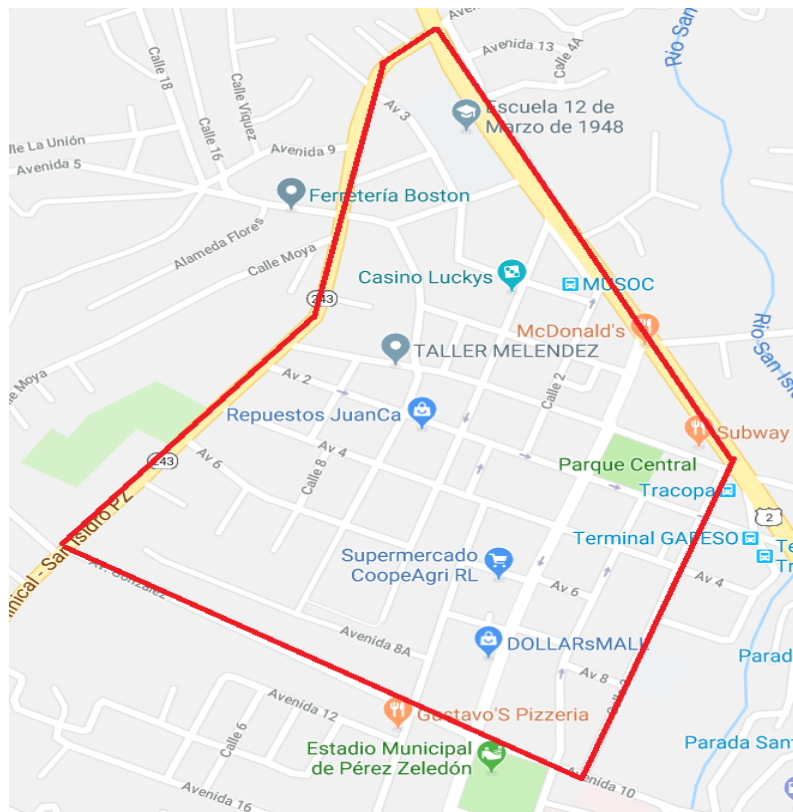
DESCRIPCIÓN DEL PROYECTO #2

“RUTAS DE TRANSPORTE 1.0”

Descripción

En una ciudad moderna de nombre vamos a decir que se llama “La Mancha”, el sistema de transporte público debe mejorarse, por ese motivo es que se ha establecido la necesidad de manejar más información referente a rutas de autobuses, estaciones, conexiones entre paradas y registros de uso de dichos medios de transporte.

Para apoyar a la administración municipal (que lo requiere y en mucho la ayuda), se solicita a los estudiantes del curso de Estructuras de Datos, desarrollar una aplicación que permita gestionar y analizar la red de transporte actual y por medio de análisis de algoritmos el tratar de optimizar dichas rutas.



Figura#1: Mapa del transporte público con fines ilustrativos

Requerimientos mínimos:

1. Uso de Árboles:

- Implementar un árbol binario de búsqueda para almacenar información de las estaciones y paradas de autobuses de la ciudad “La Mancha” (ejemplo: número de identificación como clave, junto con nombre).
- El árbol debe permitir inserción, búsqueda, eliminación y recorridos (entre-orden, pre-orden, post-orden) entre las distintas paradas o estaciones de autobuses. Los recorridos deben mostrarse y exportarse a archivos (recorridos_rutas.txt).
- Permitir consultas como: mostrar todas las estaciones de autobuses que están sobre una ruta. Generarse según el algoritmo seleccionado por el usuario del sistema.

2. Uso de Grafos (Interfaz gráfica):

- Representar la red de transporte mediante un grafo ponderado (Figura #1):
 - i. Los nodos representan las paradas o estaciones.
 - ii. Las aristas representan los tramos entre estaciones, con un peso que indique el tiempo estimado de viaje.
 - iii. La ruta de transporte es el recorrido completo entre los tramos de estaciones seleccionadas.
 - iv. Los pesos (tiempo) puede variar (aumentar o disminuir) según se determine al iniciar el viaje, ya que pueden registrarse cierres de vía o accidentes. Ocasionando que se cambie de ruta (primer caso) o se aumente el tiempo (segundo caso).
- Debe implementarse:
 - i. Algoritmos de recorrido en amplitud (BFS) y profundidad (DFS).
 - ii. Algoritmo de búsqueda de caminos más cortos (Dijkstra y Floyd-Warshall).
 - a. Dijkstra: al aplicar este algoritmo se deben seleccionar dos vértices (estaciones) y el algoritmo generará todos los caminos más cortos entre el primer vértice a todos los demás, pero únicamente se utiliza en pantalla el camino más corto entre los dos vértices seleccionados.
 - b. Floyd: este algoritmo construye una matriz de costos mínimos y de caminos. Se debe utilizar únicamente el camino más corto para determinar la ruta de la estación A a la estación B.
 - iii. Algoritmo de expansión mínima (Kruskal y Prim).
- Ejemplo de consulta: calcular y mostrar la ruta más corta entre dos estaciones dadas (solicitadas por el usuario) y seleccionando el algoritmo respectivo (punto anterior).

3. Manejo de Archivos:

- Uso de archivos para mantener la información entre las distintas ejecuciones del programa.
- Archivos recomendados:

- i. estaciones.txt // para almacenar información de las estaciones o paradas de autobuses presentes en la ciudad.
 - ii. rutas.txt // para almacenar la red de transporte (nodos y conexiones).
 - iii. reportes.txt // para generar informes de uso (por ejemplo, número de estaciones, consultas de rutas, historiales).
 - Se debe permitir cargar y guardar datos de manera automática al iniciar/cerrar el programa. Deben existir las opciones en pantalla requeridas (botones asociados) para tales acciones.
4. Funcionalidades adicionales:
- Registrar nuevas estaciones o paradas de autobuses en el árbol.
 - Eliminar paradas o estaciones que ya no se utilizarán en la ciudad.
 - Consultar la ruta más corta entre dos paradas o estaciones (incluir la selección del algoritmo).
 - Debe marcarse cierre de vías como cerradas (cargar un archivo llamado cierres.txt. que tendrán los tramos de las rutas cerradas).
 - En caso de cierre de vías (ya que en la ciudad de “La Mancha” se están haciendo trabajos de alcantarillado). Los algoritmos deben redireccionar la mejor ruta.
 - Generar reportes:
 - i. Visualizar rutas y conexiones: mostrar todas las rutas existentes en el árbol entre los puntos dados por el usuario.
 - ii. Estaciones: ordenadas alfabéticamente usando recorridos de árboles.
 - iii. Exportar la información generada de los reportes, a un archivo llamado reportes.txt.
5. El programa debe contar con una interfaz gráfica adecuada y de fácil manejo. La pantalla debe mostrarse como en la Figura #1.

Instrucciones Generales

1. El trabajo deberá entregarse en forma digital (fuentes y ejecutable) el día y hora indicados en el aula virtual según se indica en el cronograma del curso. El proyecto será desarrollado de **forma individual**.
2. El proyecto será desarrollado en la herramienta **VS Community 2022 - C++**.
3. El proyecto deberá llevar su control con una herramienta de control de versiones elegida por el profesor a cargo del curso (GitHub). Para ello se hará una revisión de avance según las fechas establecidas.
4. El proyecto será probado en dos ocasiones, la primera será con el profesor y los estudiantes y la segunda será por parte del profesor a cargo del grupo.

Rúbrica de la Evaluación del Proyecto

Criterio	Puntaje
Funcionalidad	40%
Implementación de Arboles	13%
Implementación de Grafos	12%
Manejo de Archivos	10%
Interfaz de Usuario y Usabilidad	10%
Creatividad y Complejidad	10%
Defensa	5%
TOTAL	100%

Fecha de Entrega

Entrega del enunciado: Jueves 18 de setiembre del 2025.
Revisión y defensa final: Jueves 13 de noviembre del 2025.

Anexos

Anexo #1

Buenas Prácticas solicitadas en el proyecto

Código
Utiliza estándares propios del lenguaje C++.
Utiliza control de versiones Git
Utiliza múltiples commits para la realización del proyecto, se denota una planeación en la realización del proyecto (Existe una cantidad de commits y orden razonable)
Cada commit corresponde a una tarea única, está bien comentado en inglés. (buena documentación)
Utiliza código correctamente formateado, escrito en inglés.
El código de cada archivo esta ordenado, variables declaradas primero, luego métodos públicos (empieza con el constructor y destructor) y de ultimo métodos privados
Archivos
Maneja archivos separados para cada estructura o clase, así como de encabezado.
El nombre de cada archivo cumple con las buenas prácticas: extensiones correctas, nombre representativo, utiliza los caracteres correctos.
Nomenclatura
El nombre de cada clase cumple con las buenas prácticas: nombre representativo, PascalCase, utiliza los caracteres correctos, es un sustantivo, está en inglés.
El nombre de cada atributo, variable o parámetro cumple con las buenas prácticas: nombre representativo, no incluye tipo, pero representa su identidad (por ejemplo, booleanos positivos, o nombres plurales), camelCase, utiliza los caracteres correctos, es un sustantivo, está en ingles
El nombre de cada método o función cumple con las buenas prácticas: nombre representativo, camelCase, utiliza los caracteres correctos, inicia con un verbo, indica correctamente lo que devuelve, realiza función única, muestra de manera explícita lo que debe recibir, está en ingles
Estructuras de control
Utiliza curly brackets siempre
Utiliza los diferentes ciclos para lo que es
Evita anidaciones de más de 3 en cada función
La lógica booleana esta optimizada, prefiere expresiones de condición legibles, utiliza el else solo cuando es necesario, utiliza mejores alternativas al switch
Mensajes al usuario y comentarios
El código no contiene comentarios
Los mensajes al usuario son positivos, están en español, tienen buena gramática y ortografía
Programación orientada a objetos

	Utiliza herencia cuando es necesario
	Utiliza polimorfismo cuando es necesario
	Utiliza un nivel de abstracción adecuado
	Utiliza encapsulamiento (abierto a extensión, cerrado para modificación)
	Utiliza bajo acoplamiento y alta cohesión
	Utiliza el principio de responsabilidad única
	Utiliza el principio de DRY, reutiliza código, evita repetirlo